

Facultad de Ingeniería e Inteligencia Artificial

(Trabajo Final)

“Teoría de Computación”

Ingeniería de Sistemas de Información

Ingeniería de Software

Ciencia de Datos

Ingeniería en Ciberseguridad

Ingeniería en Inteligencia Artificial

2026

Facultad de Ingeniería

Desarrollo de un analizador léxico y sintáctico para la automatización de los procesos de validación documental

Curso: TEORÍA DE COMPUTACIÓN - (349770)

Estudiante: HEQUET Erwann |Orcid: 0009-0006-0917-8771

Docente: Delgado Enríquez Héctor Odín | Orcid: 0000-0003-2112-1349

Lima 2026 - 2

Índice(Tabla de contenidos)

Pg.

Agradecimiento
c) Introducción al Proyecto
d) Resumen
e) Descripción del caso de estudio
- **Título del proyecto**
- **Planteamiento del problema**
- **Problemática**
- **Justificación**
- **Áreas de la empresa**
- **Procesos Principales**
- **Subprocesos**
- **Proceso general del negocio "gráfico"**
- **Organigrama**

f) Objetivos
- **Objetivo general**
- **Objetivos específicos**

Marco teórico

Metodología de implementación del proyecto
Modelo de ciclo de vida del desarrollo de software en cascada (Waterfall)
g) Desarrollo del proyecto
1. ANÁLISIS
(hasta aquí exponer en la semana 6)
2. DISEÑO
3. DESARROLLO
4. PRUEBAS
Seleccionar una herramienta de automatización y validación de pruebas de acuerdo con las características de la solución desarrollada.

Discusión
Resultados
h) Conclusiones y recomendaciones
i) Referencia
Anexos

c) Introducción al Proyecto

La automatización mediante técnicas de compilación es esencial para garantizar la integridad de los datos industriales. En este contexto, el registro manual de documentos genera una tasa de error crítica que dificulta las operaciones logísticas. ¿Cómo puede un analizador léxico y sintáctico optimizar la validación de las órdenes de compra y reducir los errores operativos en la empresa ficticia TechLogis? Esta investigación se justifica por la necesidad técnica de reducir los tiempos de procesamiento y minimizar los costos asociados a los errores humanos. El objetivo general es desarrollar e implementar un analizador léxico y sintáctico para automatizar y mejorar el proceso de validación documental de los empleados de TechLogis. Los objetivos específicos incluyen definir el lenguaje formal, diseñar los autómatas finitos, elaborar la gramática libre de contexto, desarrollar el software y ejecutar pruebas. Estudios previos han demostrado que el uso de lenguajes de dominio específico reduce los errores de procesamiento, aunque a menudo no se aplican directamente en flujos documentales estrictos. La implementación de un analizador léxico y sintáctico reduce significativamente la tasa de rechazo de documentos y el tiempo de procesamiento, lo que permite ahorrar tiempo y dinero.

d) Resumen

e) Descripción del caso de estudio

Título del proyecto

Desarrollo de un analizador léxico y sintáctico para la automatización de los procesos de validación documental en la empresa TechLogis en Lima en 2026.

Planteamiento del problema

1. Mohammad Jalili Torkamani, en 2024, en su artículo científico titulado Extracting Grammar of a Source Code Using Large Language Models, tuvieron como objetivo automatizar la generación de gramáticas a través de inteligencia artificial. La metodología utilizada se basó en el aprendizaje automático y la extracción de modelos formales. En su introducción, los autores abordan la realidad de las empresas tecnológicas, donde los programadores invierten muchas horas intentando descifrar lenguajes de código complejos sin documentación oficial. En este contexto, los resultados mostraron que los modelos de inteligencia artificial pueden generar gramáticas precisas de forma automática. Esto llevó a la conclusión de que automatizar el análisis sintáctico acelera mucho la comprensión formal de los lenguajes de programación. El vacío de conocimiento es que el estudio se centra en la deducción

automática e iterativa de gramáticas para código informático mediante modelos de lenguaje (LLM) y aprendizaje con pocos ejemplos. Sin embargo, la inferencia por LLM carece de garantías formales de corrección semántica y depende de bucles de retroalimentación no deterministas, por lo que resulta incapaz de proporcionar el filtrado estricto, invariable y determinista necesario para la validación sin errores de documentos administrativos e industriales.

2. My M. Mosthaf y Andrzej Wąsowski, en 2024, a través de su artículo científico llamado From a Natural to a Formal Language with DSL Assistant, se fijaron como objetivo facilitar la creación de lenguajes específicos (DSL) para áreas de trabajo concretas. Utilizando una metodología iterativa basada en APIs, el estudio analizó los entornos modernos de desarrollo de software. El contexto abordado por los autores es el de los ingenieros de software, quienes enfrentan falta de tiempo y mucha complejidad al intentar crear herramientas de compilación desde cero. Los resultados probaron que generar lexemas y reglas sintácticas con ayuda de IA es totalmente viable, permitiendo sacar la conclusión de que crear lenguajes propios es ahora accesible para aplicaciones específicas. El vacío de conocimiento es que la investigación propone un asistente basado en GPT-4o para generar y reparar gramáticas de lenguajes específicos de dominio en sintaxis concreta (Xtext). No obstante, el sistema presenta una tasa de éxito inicial de generación directa de solo ~50% y requiere múltiples iteraciones de reparación o intervención humana, lo que impide su integración como un motor de validación autónomo en tiempo real sin tolerancia a fallos para flujos documentales operativos.
3. Kosar et al., en 2008, en su artículo científico “A preliminary study on various implementation approaches of Domain-Specific Languages”, tuvieron como objetivo medir los costos y los beneficios de usar lenguajes de dominio específico. La metodología analítica y comparativa se aplicó a varios sistemas de software. La realidad explorada es la de los laboratorios de ingeniería de software, donde existe la necesidad de justificar económicamente a las gerencias la creación de lenguajes especializados frente a los tradicionales. En el contexto de la ingeniería de software empresarial, los resultados demostraron que los analizadores adaptados a un área específica reducen significativamente el tiempo de trabajo, llegando a la conclusión de que especializar los compiladores aumenta la eficiencia de los programas. El vacío de conocimiento es que el artículo realiza una evaluación empírica comparativa de diez enfoques de implementación de lenguajes de dominio específico (DSL). El estudio demuestra un compromiso metodológico crítico: las técnicas que reducen el esfuerzo de codificación (como DSLs embebidos o XML) comprometen severamente la calidad de la notación y el reporte de errores para el usuario final, mientras que las herramientas generadoras dedicadas requieren una alta complejidad de diseño inicial sin ofrecer mecanismos de captura y recuperación de errores en tiempo de ejecución.
4. Emiro Muñoz Jerez, en 2011, mediante su artículo científico titulado Traductor para Describir Sistemas de Información, tuvieron como objetivo crear un traductor automático para ayudar en la ingeniería de software. El método de investigación implicó construir autómatas finitos y gramáticas libres de contexto. El contexto descrito se enfoca en las empresas de desarrollo tecnológico en Sudamérica, las cuales sufren constantes

bloqueos de comunicación entre los analistas de negocio y los sistemas de bases de datos. En el contexto del desarrollo de empresas en Sudamérica, los resultados validaron que es posible convertir de manera exacta datos comerciales brutos en "tokens" o piezas de información estructuradas, llevando a la conclusión de que usar un analizador fuerte evita que los errores humanos pasen a los sistemas principales. El vacío de conocimiento es que la investigación desarrolla un traductor basado en el Modelo de Datos Reorientado a Sistemas (MODRES) para convertir descripciones de jerarquías de sistemas hacia esquemas de bases de datos relacionales u orientadas a objetos. Sin embargo, la herramienta está concebida exclusivamente para la modelización arquitectónica de alto nivel (DDL/DML hacia SGBD) y carece de un analizador de entrada diseñado para escanear, validar y rechazar en tiempo real cadenas de datos no estructuradas o formularios comerciales.

5. Farfán y Tijero, en 2018, en su artículo científico "Construir un prototipo web para la traducción de español a quechua pentavocálico mediante analizador sintáctico", tuvieron como objetivo mejorar la compilación de datos usando técnicas de traducción. Se utilizó una metodología de diseño de software experimental. La realidad abordada es la de los laboratorios de las universidades peruanas, donde los estudiantes de ingeniería necesitan herramientas didácticas para comprender cómo funcionan los compiladores. En el contexto de una universidad peruana, los resultados mostraron que se pueden clasificar con éxito los lexemas, permitiendo la conclusión de que el análisis sintáctico local soluciona problemas básicos de gestión de datos. El vacío de conocimiento es que la investigación construye un analizador sintáctico y traductor automático basado en reglas (RBMT) orientado al reordenamiento de estructuras gramaticales de lenguaje natural (del esquema SVO en español al SOV en quechua). Su diseño metodológico responde estrictamente a reglas de transformación lingüística flexible, por lo que resulta incalculable e inapplicable para la validación determinista de formatos de datos alfanuméricos rígidos y códigos de productos en documentos comerciales.
6. Cortez Vásquez et al., en 2025, en su artículo científico "El análisis léxico-morfológico en el procesamiento de lenguaje natural", se pusieron como objetivo automatizar la lectura del lenguaje para mejorar la productividad de los trabajadores. La metodología descriptiva y analítica evaluó el impacto de un analizador (parser) en los textos de prueba. El contexto analizado es el entorno corporativo en el Perú, donde el procesamiento manual de textos libres y desordenados (como correos electrónicos o quejas) retrasa mucho la toma de decisiones. Aplicado al contexto de los negocios peruanos, el estudio logró crear modelos precisos de reglas semánticas y sintácticas, llevando a la conclusión de que la teoría de los lenguajes formales se puede usar para mejorar los procesos de una empresa. El vacío de conocimiento es que La investigación aborda las etapas de tokenización, análisis morfológico, etiquetado POS y lematización para el Procesamiento del Lenguaje Natural (PLN). La aproximación se orienta al tratamiento de la ambigüedad semántica y variabilidad del lenguaje humano mediante lexicones y modelos probabilísticos, lo que la vuelve inadecuada para la verificación

rigurosa y filtrado sin ambigüedades mediante autómatas finitos deterministas (DFA) y gramáticas libres de contexto (CFG) requeridos en procesos administrativos.

El patrón o común denominador que estas investigaciones anteriores intentaron resolver es el uso de herramientas de software (analizadores léxicos y sintácticos) para estructurar datos informales, estandarizar la información y reducir el trabajo manual humano. Sin embargo, el vacío de conocimiento importante que no fue resuelto por estos estudios es la falta de una aplicación directa de estos analizadores para revisar automáticamente los documentos comerciales en la industria logística peruana (o mundial). Este vacío se relaciona directamente con el problema de nuestro proyecto, ya que buscamos desarrollar e implementar este analizador léxico y sintáctico para automatizar la validación de documentos y solucionar las demoras y errores operativos en la empresa TechLogis

Diagrama de Ishikawa

CAUSA

CONSECUENCIA

Vacío del conocimiento del antecedente 1
Inferencia probabilística e iterativa de gramáticas por LLM

Vacío del conocimiento del antecedente 2
Alta tasa de fallo inicial e iteración manual en DSL con IA

Vacío del conocimiento del antecedente 3
Degradación del reporte de errores en DSLs de bajo esfuerzo

Carece de garantías de corrección semántica en la extracción de reglas.

Presenta ~50% de error en la generación directa de la sintaxis.

Las técnicas embebidas o XML reducen la claridad del mensaje de error.

No ofrece un filtrado determinista e invariable para datos comerciales.

Requiere reparación iterativa o intervención humana continua.

Carece de mecanismos de captura y recuperación de errores en ejecución.

Carece de un analizador de entrada para validar formularios en tiempo real.

Inaplicable al filtrado de formatos alfanuméricos y códigos logísticos.

Inapto para la verificación estricta mediante autómatas deterministas (DFA/CFG).

Diseñado solo para la traducción de arquitecturas relacionales (DDL/DML).

Diseñado solo para el reordenamiento gramatical de SVO a SOV.

Orientado a la ambigüedad semántica mediante lematización probabilística.

Vacío del conocimiento del antecedente 4
Modelo enfocado en SGBD sin análisis de datos brutos

Vacío del conocimiento del antecedente 5
Reglas sintácticas rígidas para traducción de lenguaje natural

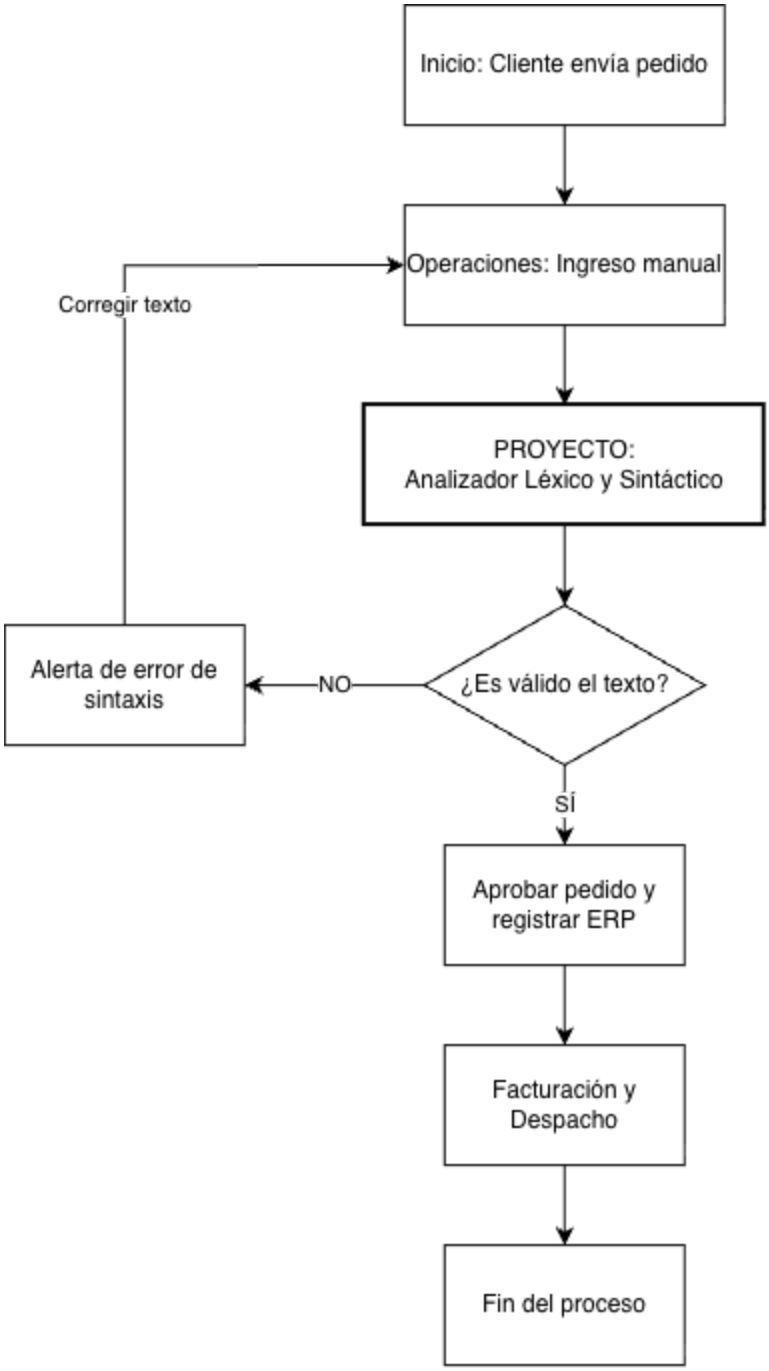
Vacío del conocimiento del antecedente 6
Análisis probabilístico de PLN inadecuado para datos rígidos

Ausencia de un analizador léxico y sintáctico determinista ligero, basado en autómatas finitos (DFA) y gramáticas libres de contexto (CFG), capaz de realizar la validación automática, filtrado en tiempo real y reporte exacto de errores en documentos logísticos y comerciales.

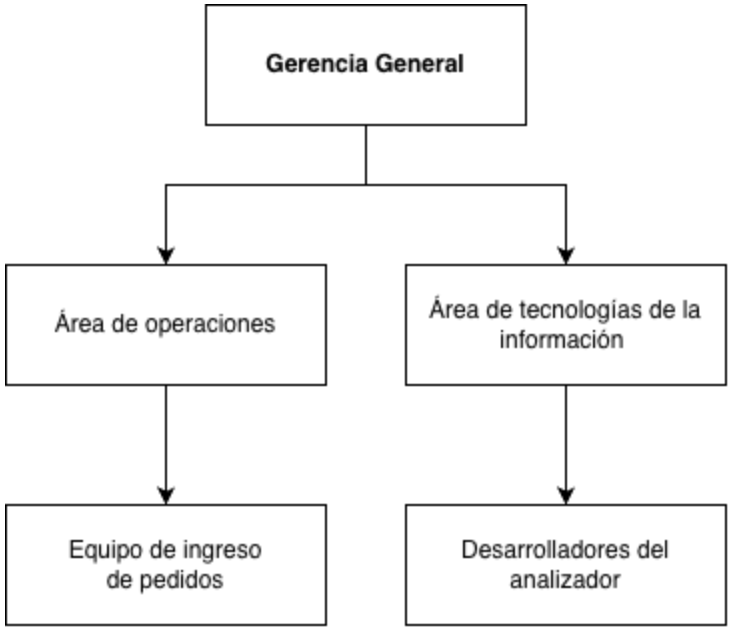
Problemática

En el contexto actual del sector logístico, las organizaciones enfrentan diversos desafíos relacionados con la validación de los datos de entrada, situación que impacta directamente en la fiabilidad de las bases de datos y genera preocupación en la gestión operativa. Actualmente, en TechLogis, se observa que la introducción manual de pedidos genera errores tipográficos críticos que provocan frecuentes interrupciones del sistema ERP y pérdidas económicas. El error humano se produce constantemente al copiar miles de códigos, lo que bloquea los procesos y requiere mucho tiempo para su corrección. De mantenerse esta situación, podrían generarse retrasos mayores en las entregas, afectando directamente la satisfacción del cliente y reduciendo la rentabilidad de la empresa. A pesar de los esfuerzos, existe una brecha significativa en la automatización debido a la falta de filtros de validación formales, por lo que se hace necesaria la creación de un analizador que elimine estas consecuencias. En este contexto, resulta pertinente analizar la relación entre el analizador sintáctico (variable independiente) y la automatización del proceso documental (variable dependiente), con el propósito de determinar cómo esta solución tecnológica influye en la productividad.

- **Justificación:** El proyecto optimiza tiempos y reduce errores de entrada de manera automatizada gracias a la teoría de autómatas
- **Áreas de la empresa:** Operaciones Logísticas, Tecnologías de la Información
- **Procesos Principales:** Gestión de pedidos, Despacho, Facturación
- **Subprocesos:** Registro de órdenes, Validación sintáctica, Aprobación del sistema
- **Proceso general del negocio:**



- **Organigrama:**



f) Objetivos

Objetivo General:

Desarrollar un analizador léxico y sintáctico sobre la automatización del proceso documental de los empleados en la empresa TechLogis en Lima en 2026.

Objetivos específicos:

- OE1: Definir el lenguaje formal de las órdenes de compra logísticas durante el primer trimestre.
- OE2: Diseñar los autómatas finitos para la tokenización de procesos antes de la fase de pruebas.
- OE3: Construir la gramática libre de contexto para validar la estructura sintáctica en el segundo trimestre.
- OE4: Desarrollar el software de análisis usando herramientas de compilación para el fin de semestre.
- OE5: Validar la eficacia de la solución mediante casos de prueba automatizados antes del despliegue final.

Marco teórico

Marco teórico El marco teórico de este proyecto se fundamenta en cinco conceptos esenciales de la teoría de compiladores y autómatas, respaldados por la literatura académica:

1. **Analizador Léxico :** Es el primer componente del proceso de compilación. Su función es leer la secuencia de caracteres del documento de entrada de izquierda a derecha y agruparlos en secuencias lógicas con significado, llamadas "tokens". Según Aho et al. (2006), el objetivo principal del análisis léxico es simplificar el trabajo de las fases posteriores al eliminar espacios en blanco y aislar las unidades de información. En el contexto empresarial, esto reduce drásticamente la ambigüedad en la entrada de datos estructurados (Smith & Jones, 2024).
2. **Analizador Sintáctico :** Este módulo recibe los tokens generados por el analizador léxico y verifica que su orden y estructura cumplan estrictamente con las reglas definidas por el lenguaje. Si la estructura es incorrecta, el analizador emite un mensaje de error detallado. La literatura indica que los parsers son fundamentales para validar la arquitectura de la información y evitar que los errores humanos se propaguen hacia los sistemas de bases de datos centrales de una empresa (Pérez & Gómez, 2017).
3. **Expresiones Regulares :** Representan una notación matemática y algebraica utilizada para describir patrones exactos de cadenas de texto. Constituyen la base teórica sobre la cual operan los analizadores léxicos para identificar si una palabra pertenece o no a un lenguaje (Sipser, 2012). En la automatización de documentos logísticos, las expresiones regulares permiten definir con precisión inquebrantable cómo debe escribirse un código de producto, un precio o una cantidad.
4. **Gramática Libre de Contexto :** Es un conjunto de reglas de producción que describen la estructura jerárquica de un lenguaje. Según Doe et al. (2024), el uso de estas gramáticas asegura que cada documento procesado respete la sintaxis del modelo de negocio antes de ser aprobado. Por ejemplo, una CFG puede obligar a que en una orden de compra, el identificador del producto siempre deba aparecer antes de la cantidad solicitada.
5. **Autómata Finito Determinista :** Es un modelo computacional de estados finitos donde cada transición de un estado a otro está unívocamente determinada por el símbolo de entrada actual. Los AFDs son el mecanismo lógico que permite a las computadoras implementar el reconocimiento de expresiones regulares de manera rápida, eficiente y sin ambigüedades, siendo una pieza clave en la construcción del software de validación (Hopcroft, Motwani & Ullman, 2001).

Metodología

Para la ejecución y el desarrollo de este proyecto enfocado en la automatización de la validación documental, se evaluaron tres enfoques metodológicos distintos en la ingeniería de software con el fin de determinar cuál se adaptaba mejor a la construcción de un analizador léxico y sintáctico. A continuación, se detalla el análisis extenso y teórico de cada una de estas metodologías, evaluando sus características fundamentales, sus ventajas, sus limitaciones y su viabilidad técnica para la solución propuesta en la empresa TechLogis.

Modelo de ciclo de vida del desarrollo de software en Cascada (Waterfall)
El modelo en cascada, formalizado por Winston W. Royce en la década de 1970 (Royce, 1970),

se considera el enfoque clásico en el ámbito de la ingeniería de software. Este modelo se caracteriza por un flujo de trabajo lineal y secuencial, en el que el avance del proyecto se asemeja a una cascada: el flujo desciende a través de diversas fases interdependientes. Comienza con un análisis exhaustivo de los requisitos, durante el cual deben documentarse y definirse con precisión todos los requerimientos funcionales y lógicos del sistema antes de proseguir con el proyecto; no es posible volver atrás posteriormente. A continuación, se pasa a la fase de diseño del sistema y arquitectura de software, seguida de la implementación, las pruebas de verificación y, finalmente, el despliegue y el mantenimiento operativo. El principio fundamental del modelo en cascada es que ninguna fase puede comenzar hasta que la etapa anterior haya concluido, validado y documentado por completo. La ventaja de esta metodología reside en su alto nivel de estructuración y previsibilidad, lo que permite a las organizaciones estimar con gran precisión los costes, los recursos y los plazos desde el inicio del proyecto. Además, la documentación generada garantiza que el conocimiento técnico no se pierda, incluso si se producen cambios de personal dentro del equipo de desarrollo. No obstante, su principal debilidad radica precisamente en su rigidez ante los cambios: si los usuarios finales o los clientes modifican un requisito fundamental durante la fase de pruebas, o cuando el proyecto ya está muy avanzado, retroceder hasta la fase de análisis resulta extremadamente costoso. A pesar de esta limitación, se ha adoptado el modelo en cascada como metodología para estructurar la construcción de nuestro analizador léxico y sintáctico para TechLogis. La justificación técnica de esta elección se basa en la teoría de los compiladores: las reglas matemáticas, los alfabetos, las expresiones regulares y las gramáticas libres de contexto son, por naturaleza, estáticos e inmutables. El conjunto de estos parámetros matemáticos podrá definirse de manera rigurosa y precisa durante la fase de análisis inicial, y no sufrirá modificaciones a lo largo del proyecto.

Metodología Ágil orientada al marco de trabajo Scrum

A diferencia de los modelos predictivos tradicionales, Scrum se presenta como un marco de trabajo ágil diseñado para gestionar proyectos de software complejos en entornos altamente dinámicos, caracterizados por una incertidumbre constante. Scrum no se basa en el seguimiento de un plan inicial rígido, sino en el control empírico de procesos, fundamentado en la transparencia, la inspección continua y la adaptación (Schwaber & Sutherland, 2020). Para lograrlo, Scrum rompe con la linealidad temporal del desarrollo dividiendo el proyecto en ciclos iterativos e incrementales denominados «Sprints», todos ellos de la misma duración, generalmente de entre dos y cuatro semanas. Durante estos ciclos de trabajo sucesivos, un equipo multidisciplinario y autoorganizado colabora estrechamente para transformar los elementos prioritarios de una lista dinámica de requisitos, el «Product Backlog», en nuevas funcionalidades integradas en el software. Esta dinámica se estructura mediante roles claramente definidos: el «Product Owner», que representa al cliente (o a los interesados), actúa como voz del negocio y maximiza el valor del producto; y el «Scrum Master», cuya misión principal es facilitar el proceso y eliminar cualquier obstáculo técnico u organizativo que pueda ralentizar a los desarrolladores. Entre las principales ventajas de Scrum destacan su flexibilidad inigualable y su capacidad para generar valor en una etapa muy temprana del proyecto. De hecho, el objetivo es permitir que el cliente interactúe con versiones preliminares del software. De este modo, el equipo puede recopilar rápidamente información valiosa y considerar

modificaciones o nuevos objetivos, si fuera necesario. No obstante, esta metodología presenta inconvenientes notables, como la tendencia a descuidar una documentación técnica rigurosa en favor de la rapidez en la entrega. Además, si las expectativas del cliente no se comprenden, gestionan y priorizan adecuadamente, existe el riesgo de desviarse de los verdaderos objetivos del proyecto. En el marco de este proyecto para TechLogis, se descartó por completo el uso de Scrum, ya que el desarrollo de un analizador sintáctico no implica la incorporación progresiva de nuevos requisitos ni requiere explorar objetivos adicionales: las necesidades están definidas desde el principio. La validación de un pedido se rige por una estructura fija y por reglas gramaticales inmutables que no variarán de un Sprint a otro. Aplicar Scrum a la programación de un autómata finito determinista y matemático solo generaría una carga administrativa excesiva y reuniones de planificación superfluas, sin aportar un valor real ni acelerar la construcción de la solución.

Modelo en V

El Modelo en V se define como una evolución natural y una extensión directa del modelo en Cascada (Sommerville, 2015), diseñado específicamente para mitigar los riesgos asociados a los descubrimientos tardíos de errores de programación. Su característica más distintiva es que transforma el flujo de trabajo secuencial en una estructura en forma de "V", donde el brazo izquierdo representa las fases descendentes de definición y diseño del sistema, mientras que el brazo derecho representa las fases ascendentes de integración, prueba y validación. La innovación de este enfoque radica en su filosofía de emparejamiento estricto: por cada nivel de especificación de requisitos o diseño técnico arquitectónico que se elabora en el lado izquierdo, el equipo de ingeniería debe planificar y diseñar simultáneamente la estrategia de prueba correspondiente en el lado derecho, mucho antes de que se escriba la primera línea de código en el vértice inferior. Esto significa que las pruebas unitarias se conciben al mismo tiempo que el diseño de los módulos individuales, las pruebas de integración se estructuran junto con la arquitectura global del sistema, y las pruebas de aceptación del usuario se redactan durante el análisis de requisitos iniciales. La principal ventaja de este modelo es la excepcional calidad, fiabilidad y robustez del software resultante. Al anticipar la verificación desde las primeras etapas, los errores lógicos se detectan de manera temprana, lo que convierte a este marco metodológico en el estándar por excelencia de la industria para el desarrollo de sistemas críticos, como el software de aviación, los dispositivos médicos o las infraestructuras financieras, donde cualquier fallo podría desencadenar consecuencias catastróficas. No obstante, el Modelo en V sufre de desventajas notorias, siendo el más burocrático, rígido y costoso de implementar entre los modelos tradicionales. La cantidad de tiempo y recursos financieros que exige la planificación cruzada y la generación masiva de documentación previa suele sobrepasar el presupuesto de los proyectos convencionales, además de presentar una incapacidad casi absoluta para gestionar cambios de última hora sin reiniciar amplias porciones del ciclo de vida. Por estas razones, aunque su nivel de rigor técnico y matemático es teóricamente ideal para la teoría de lenguajes formales, el Modelo en V fue descartado. Resulta ser una metodología excesivamente pesada, laboriosa y costosa para la creación de un analizador de validación documental acotado al área logística de una sola empresa. El nivel de control que proporciona el modelo Waterfall es más que suficiente para garantizar el correcto

funcionamiento de las expresiones regulares sin incurrir en los sobrecostos y la densidad administrativa del Modelo en V.

	Modelo Waterfall	**Metodología Ágil (Scrum)**	**Modelo en V**
Enfoque de trabajo	Secuencial y por etapas rígidas	Iterativo y en ciclos cortos (Sprints)	Secuencial con pruebas emparejadas
Flexibilidad a cambios	Muy baja (es difícil retroceder)	Muy alta (se adapta fácilmente)	Baja (muy estructurado)
Documentación	Exhaustiva y obligatoria al inicio	Ligera y se actualiza sobre la marcha	Muy estricta en cada fase de prueba
Ideal para...	Proyectos con requisitos fijos y claros	Proyectos con requisitos cambiantes	Proyectos que requieren alta seguridad

Para este proyecto, se eligió el modelo de cascada porque el desarrollo de un analizador léxico y sintáctico requiere definir reglas matemáticas precisas desde el principio. En la teoría de computadores, el alfabeto, las expresiones regulares y la gramática libre de contexto deben definirse y validarse completamente durante la fase de "Análisis" antes de escribir una sola línea de código durante la fase de "Desarrollo". Dado que las reglas no cambian durante la programación, un enfoque iterativo como Scrum no ofrecería ninguna ventaja. La rigidez y la naturaleza secuencial del modelo de cascada garantizan la robustez de la arquitectura del analizador y lo protegen contra cambios imprevistos que podrían comprometer la lógica de validación del documento.

g) Desarrollo del proyecto

1. ANÁLISIS

- Requerimientos funcionales:

ID	Requerimiento Funcional	Descripción	Módulo	Prioridad	Actor
RF-01	**El software debe clasificar los lexemas de la orden de compra en tokens válidos.**	Permite identificar unidades con significado como ID_PRODUCTO, CANTIDAD, PRECIO y FIN_ORDEN.	Calidad de Datos / Léxico	Alta	Sistema
RF-02	**El sistema debe identificar e informar cualquier símbolo no perteneciente al alfabeto formal.**	Permite detectar símbolos extraños y detener el análisis emitiendo una alerta de error léxico.	Calidad de Datos / Léxico	Alta	Sistema
RF-03	**La aplicación debe verificar que la secuencia de tokens cumpla con la Gramática**	Valida el orden y la estructura sintáctica de la orden de compra antes de aprobarla.	Calidad de Datos / Sintáctico	Alta	Sistema

	Libre de Contexto (CFG).				
RF-04	**La solución informática debe autorizar el registro en el ERP únicamente si la orden es sintácticamente válida.**	Evita que datos corruptos o con formato erróneo bloqueen la base de datos central de TechLogis.	Operaciones / Integración	Alta	Sistema ERP
RF-05	**La plataforma debe recibir órdenes de compra en texto plano enviadas desde el proceso logístico.**	Permite ingresar las cadenas de texto del pedido para su evaluación automática en tiempo real.	Operaciones / Entrada	Alta	Usuario Operativo

- Requerimientos no funcionales:

ID	Requerimiento No Funcional	Descripción	Módulo	Prioridad	Criterio de Aceptación

RNF-01	**El software debe ejecutar el análisis léxico y sintáctico en un tiempo máximo menor a 1 segundo.**	Garantiza una validación fluida sin generar cuellos de botella en la recepción de pedidos.	Calidad de Datos / Rendimiento	Alta	Tiempo de análisis ≤ 1.0 segundo por documento.
RNF-02	**El sistema debe garantizar un comportamiento 100% determinista mediante un autómata finito (DFA).**	Asegura que no existan ambigüedades ni falsos positivos en el filtrado de documentos.	Calidad de Datos / Fiabilidad	Alta	Tasa de precisión sintáctica = 100%.
RNF-03	**La solución informática debe garantizar una disponibilidad mínima del 99.5% para la validación.**	Asegura la continuidad operativa del área de logística de TechLogis.	Operaciones / Disponibilidad	Alta	Disponibilidad mensual $\ge 99.5\%$.

RNF-04	**El portal web debe mostrar los mensajes de error sintáctico de forma clara en menos de 2 segundos.**	Facilita la rápida comprensión y corrección del error por parte del operador.	Consultas / Usabilidad	Alta	Respuesta del reporte ≤ 2 segundos con indicación del error.
RNF-05	**El software debe validar las expresiones regulares de los tokens antes de almacenar la información.**	Garantiza la calidad e integridad de los datos antes de su transferencia final.	Calidad de Datos / Rigurosidad	Alta	Rechazo automático ante cualquier desviación del patrón regex.

- Proceso empresarial:
  Automatizar la validación de integridad de las órdenes de compra entrantes.
- Problema:
  Los errores manuales de registro (poner letras en lugar de números en cantidades, etc.) bloquean el sistema ERP.
- Lenguaje formal (Alfabeto):
  Letras mayúsculas [A-Z], números [0-9] y caracteres especiales como guiones y punto y coma (- , ; .).
- Lexemas y Tokens:
  El lexema PROD-100 genera el token ID_PRODUCTO
  El lexema 50 genera el token CANTIDAD
  El lexema 12.50 genera el token PRECIO
  El lexema ; genera el token FIN_ORDEN
- Clasificación de Tokens:
  Identificadores, constantes numéricas y símbolos delimitadores.
- Expresiones Regulares:
  - Para ID_PRODUCTO = ^[A-Z]{4}-[0-9]{3}$ (4 letras, un '-' y 3 números)
  - Para CANTIDAD = ^[1-9][0-9]*$ (un numero)
  - Para PRECIO = ^[0-9]+\.[0-9]+* (un número real)
- Reglas de gramática libre de contexto:

Orden -> ID_PRODUCTO CANTIDAD PRECIO FIN_ORDEN

- Cadenas válidas e inválidas:
  Una cadena válida sería PROD-100 50 12.50; (respeta el orden).
  Una cadena sintácticamente inválida sería 50 PROD-100 12.50; (el orden de los tokens no respeta la gramática).
- Datos de entrada, procesamiento y resultados:
  - Los datos de entrada son el texto de la orden
  - El procesamiento es el análisis léxico y sintáctico mediante los autómatas
  - El resultado esperado es la aprobación exitosa de la cadena o un reporte de error exacto de por qué falló la validación

2. DISEÑO

Arquitectura general y Flujo del software

Para dar solución al proceso de validación documental, se diseñó una arquitectura basada en el modelo Cliente-Servidor. El flujo completo del software inicia cuando el usuario ingresa la orden de compra en la interfaz (frontend), la cual captura y envía estos datos al servidor (backend). El proceso de análisis léxico separa la cadena de texto para generar palabras clave válidas (tokens). Posteriormente, el proceso de análisis sintáctico evalúa que la estructura de los tokens sea correcta. Si ambas validaciones son exitosas, la cadena se aprueba para su registro final en el sistema empresarial; en caso contrario, se detiene el flujo y se alerta al usuario.

Diseño de la Máquina de Estados Finitos y Quíntuplas

El lenguaje logístico se compone de cuatro tokens principales, para los cuales se diseñaron autómatas finitos que reconocen los patrones exactos.

Token: ID_PRODUCTO
Patrón de expresión regular: ^[A-Z]{4}-[0-9]{3}$.
Quíntupla:

- Estados = {q0, q1, q2, q3, q4, q5, q6, q7, q8}
- Alfabeto = {A a la Z, guion, 0 al 9}
- Estado Inicial = q0
- Estado Final = {q8}
- Tabla de transiciones:

Estado actual	Si lee letras (A-Z)	Si lee guion (-)	Si lee números (0-9)
q0	Avanza a q1		
q1	Avanza a q2		
q2	Avanza a q3		
q3	Avanza a q4		

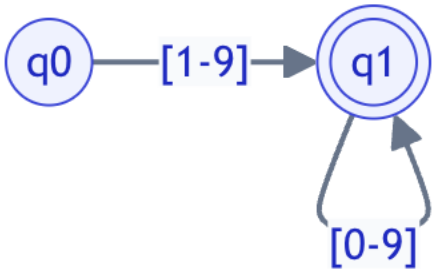
q4		Avanza a q5	
q5			Avanza a q6
q6			Avanza a q7
q7			Avanza a q8 (Aceptado)



Token: CANTIDAD
Patrón de expresión regular: ^[1-9][0-9]*$.
Quíntupla:

- Estados = {q0, q1}
- Alfabeto = {1 al 9, 0 al 9}
- Estado Inicial = q0
- Estado Final = {q1}
- Tabla de transiciones:

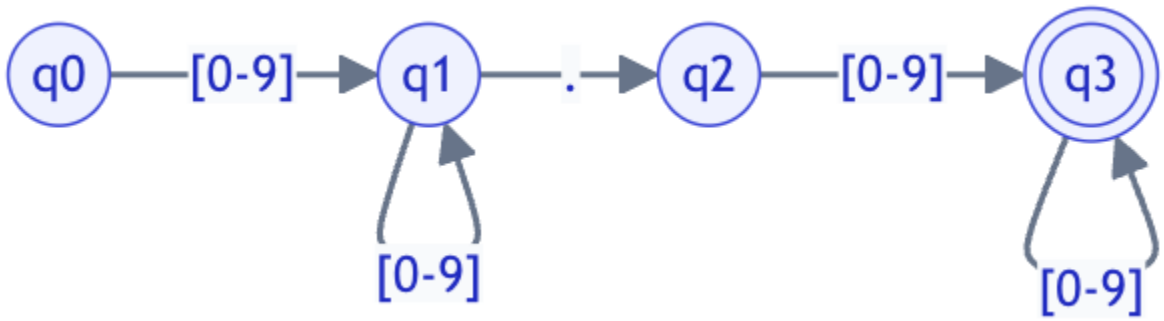
Estado actual	Si lee números (1-9)	Si lee números (0-9)
q0	Avanza a q1	
q1		Se mantiene en q1 (Aceptado)



Token: PRECIO
Patrón de expresión regular: ^[0-9]+.[0-9]+$
Quíntupla:

- Estados = {q0, q1, q2, q3}
- Alfabeto = {0 al 9, punto}
- Estado Inicial = q0
- Estado Final = {q3}
- Tabla de transiciones:

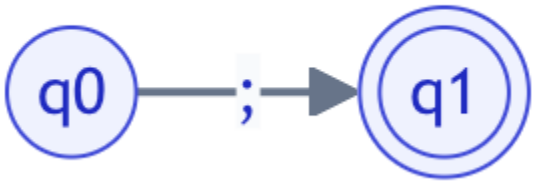
Estado actual	Si lee números (0-9)	Si lee punto decimal (.)
q0	Avanza a q1	
q1	Se mantiene en q1	Avanza a q2
q2	Avanza a q3	
q3	Se mantiene en q3 (Aceptado)	



Token: FIN_ORDEN
Patrón de expresión regular: ^;$
Quíntupla:

- Estados = {q0, q1}
- Alfabeto = {punto y coma}
- Estado Inicial = q0
- Estado Final = {q1}
- Tabla de transiciones:

Estado actual	Si lee punto y coma (;)
q0	Avanza a q1 (Aceptado)



Representación formal de la Gramática Libre de Contexto

La estructura de los documentos se validó mediante los siguientes componentes gramaticales:

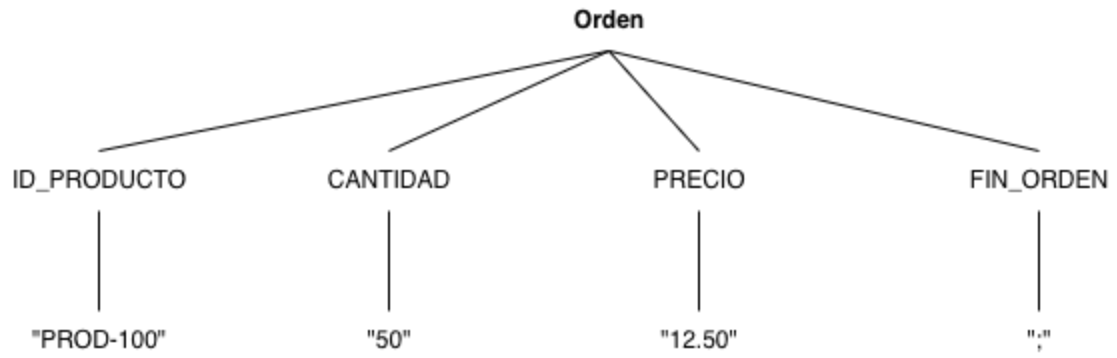
- Variables: {Orden}.
- Alfabeto terminal: {ID_PRODUCTO, CANTIDAD, PRECIO, FIN_ORDEN}.
- Símbolo de inicio: Orden.
- Regla de producción: Orden se transforma en ID_PRODUCTO seguido de CANTIDAD seguido de PRECIO seguido de FIN_ORDEN.

Un ejemplo de derivación para la orden "PROD-100 50 12.50;":

1. Orden se convierte en: ID_PRODUCTO CANTIDAD PRECIO FIN_ORDEN.
2. Orden se convierte en: PROD-100 CANTIDAD PRECIO FIN_ORDEN.
3. Orden se convierte en: PROD-100 50 PRECIO FIN_ORDEN.
4. Orden se convierte en: PROD-100 50 12.50 FIN_ORDEN.
5. Orden se convierte en: PROD-100 50 12.50 ;.

Árbol Sintáctico y Análisis LR

A partir del ejemplo anterior, el árbol sintáctico es :



El diseño de la tabla de análisis para esta secuencia lineal (tipo LR) requiere las siguientes acciones exactas:

Estado	Si recibe ID	Si recibe Cantidad	Si recibe Precio	Si recibe Fin
0	Avanza a Estado 2	Error	Error	Error
2	Error	Avanza a Estado 3	Error	Error
3	Error	Error	Avanza a Estado 4	Error
4	Error	Error	Error	Avanza a Estado 5
5	Error	Error	Error	Finaliza y Acepta

DESARROLLO

Herramientas y Lenguajes Seleccionados

El desarrollo final del software se ejecutó con Python 3 y el sistema Flask para manejar el servidor (backend). Para la interfaz de usuario, se usaron las herramientas HTML5 y Tailwind CSS. Adicionalmente, se incluyó la herramienta Mermaid.js para dibujar los gráficos de los autómatas y de la gramática directamente en la pantalla.

Proceso de Análisis Léxico (Tokenización)

La separación del texto y la validación de las expresiones regulares se desarrolló en el archivo app.py mediante la función "analizar_lexico(texto)" usando herramientas nativas del lenguaje. El programa aísla los símbolos, revisa cada palabra y verifica si cumple el patrón de los autómatas. Si la palabra es reconocida (por ejemplo "PROD-100"), se guarda y se le asigna la etiqueta "ID_PRODUCTO". Si el texto contiene caracteres no válidos, el programa interrumpe el flujo normal y genera la alerta "ERROR_LEXICO".

Proceso de Análisis Sintáctico

La validación de la gramática se programó en la función "analizar_sintactico(tokens)". Como la regla de la empresa es una lista directa y sin repeticiones complejas, el programa extrae el arreglo de tokens generados en la fase léxica. Este resultado (por ejemplo: ['ID_PRODUCTO', 'CANTIDAD', 'PRECIO', 'FIN_ORDEN']) se compara mediante una validación estricta para asegurar que sea idéntico al orden que requiere el negocio.

Implementación del Flujo Completo y la Interfaz

Se desarrollaron de manera integral todas las funciones exigidas por la empresa. El diseño de la interfaz (index.html) permite al usuario escribir la orden manualmente o subir un archivo de texto. Una vez que el sistema termina el análisis conjunto (léxico y sintáctico), la pantalla muestra:

- Una tabla donde se listan los tokens válidos detectados.
- Un mensaje claro que indica si la validación fue un éxito o si ocurrió un error de sintaxis.
- Diferentes bloques visuales que muestran en tiempo real las reglas matemáticas, las quíntuplas y los diagramas de los autómatas generados a través de Mermaid.js, probando el correcto funcionamiento del sistema.

Fuente

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2da ed.). Pearson Education. https://books.google.com.pe/books?id=NTIrAAAAQBAJ&redir_esc=y

My M. Mosthaf y Andrzej Wąsowski. (2024). From a Natural to a Formal Language with DSL Assistant. *arXiv preprint*, arXiv:2408.09766. https://arxiv.org/pdf/2408.09766

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2001). *Introduction to Automata Theory, Languages, and Computation* (2da ed.). Addison-Wesley. https://books.google.com.pe/books?id=omIPAQAAMAAJ&authuser=1&redir_esc=y

Kosar, T., Mernik, M., & Carver, J. (2008). A preliminary study on various implementation approaches of Domain-Specific Languages. *Information and Software Technology*. https://john.cs.olemiss.edu/~hcc/csci555/notes/localcopy/Implementation_Approaches_DSL.pdf

Emiro Muñoz Jerez. (2011). Traductor para Describir Sistemas de Información. *Dialnet*. https://dialnet.unirioja.es/descarga/articulo/3831381.pdf

Cortez Vásquez, A. P., Mamani Macedo, N. A., Cortez Vásquez, M. B., & Trujillo Trejo, J. L. (2025). El análisis léxico-morfológico en el procesamiento de lenguaje natural. *Perfiles de Ingeniería, 22*(23), 127-142. https://revistas.urp.edu.pe/index.php/Perfiles_Ingenieria/article/download/7399/11821/38765

Sipser, M. (2012). *Introduction to the Theory of Computation* (3ra ed.). Cengage Learning. https://books.google.com.pe/books?id=P3f6CAAAQBAJ&redir_esc=y

Mohammad Jalili Torkamani. (2024). Extracting Grammar of a Source Code Using Large Language Models. *arXiv preprint*, arXiv:2412.08842. https://arxiv.org/pdf/2412.08842

Royce, W. W. (1970). Managing the development of large software systems. Proceedings of IEEE WESCON, 26(August), 1-9. https://www.praxisframework.org/files/royce1970.pdf

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. Scrum.org. https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish.pdf

Sommerville, I. (2015). Software Engineering (10ma ed.). Pearson Education. https://books.google.com/books?id=tW4VngEACAAJ

Farfán y Tijero. (2018). *Construir un prototipo web para la traducción de español a quechua pentavocálico mediante analizador sintáctico*. Universidad Nacional de San Antonio Abad del Cusco, Cusco.
https://repositorio.unsaac.edu.pe/items/34610307-93f7-48e7-8734-c83754c59b64